COSMOS Cluster 7

Machine Learning

Summer 2026

Topic: Neural Networks

Instructors: Sam Lau and Sooh Liao

Teacher Fellow: Emmanuel Pizano

UC San Diego

Adapted from material used in ECE 20875 at Purdue University

Neural networks are everywhere

1. Image/Video/Audio recognition
2. Image/Video/Audio generation

…

Too many use cases for us to cover in this class, so we will focus on neural networks used as classifiers

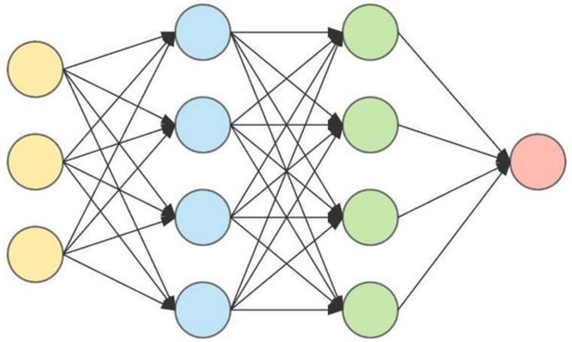
Neural networks consist of neurons

It’s similar to real/biological neurons:

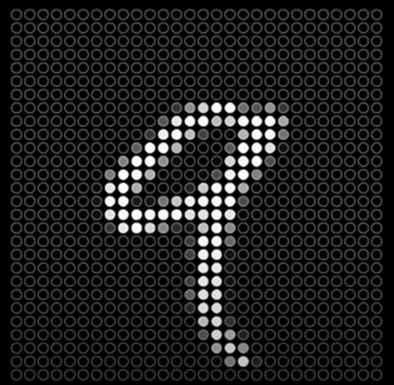
-> It takes an input signal, processes it, and outputs a new signal

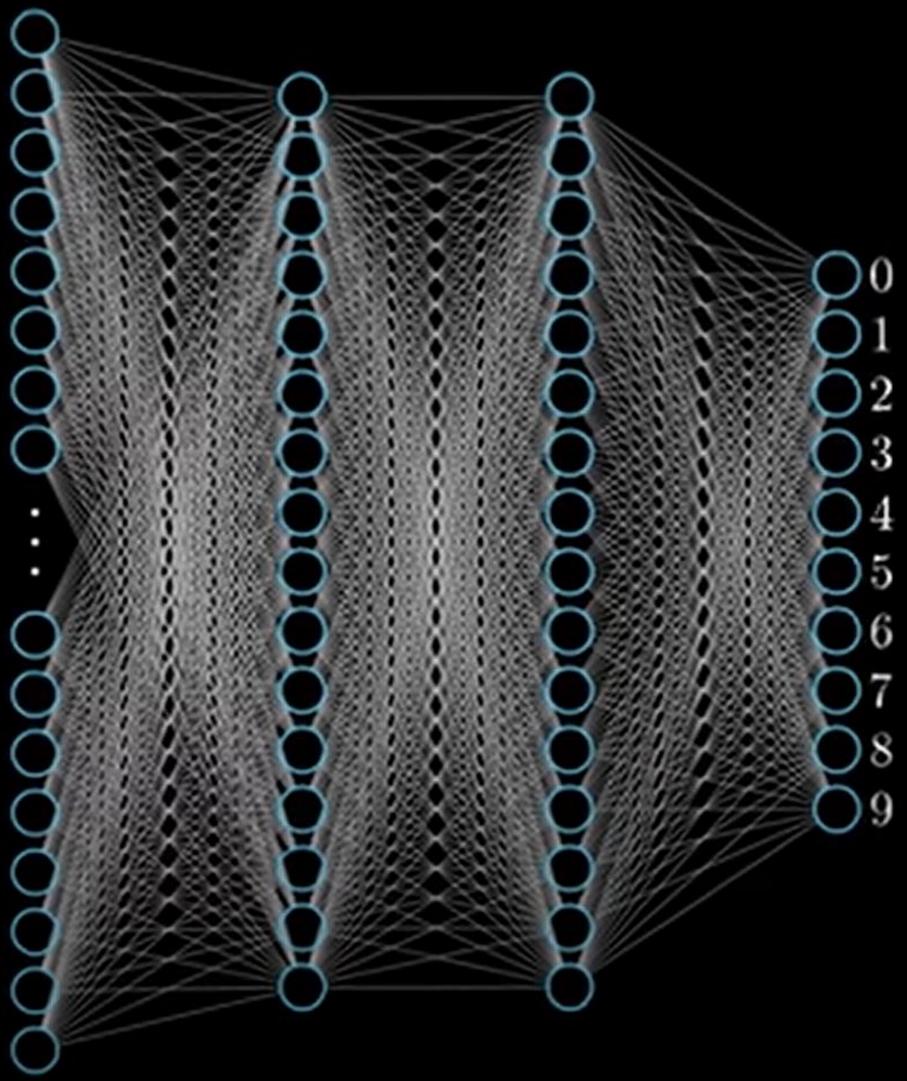
-> the output signal could also activate an adjacent neuron, if the signal is strong enough

Neurons

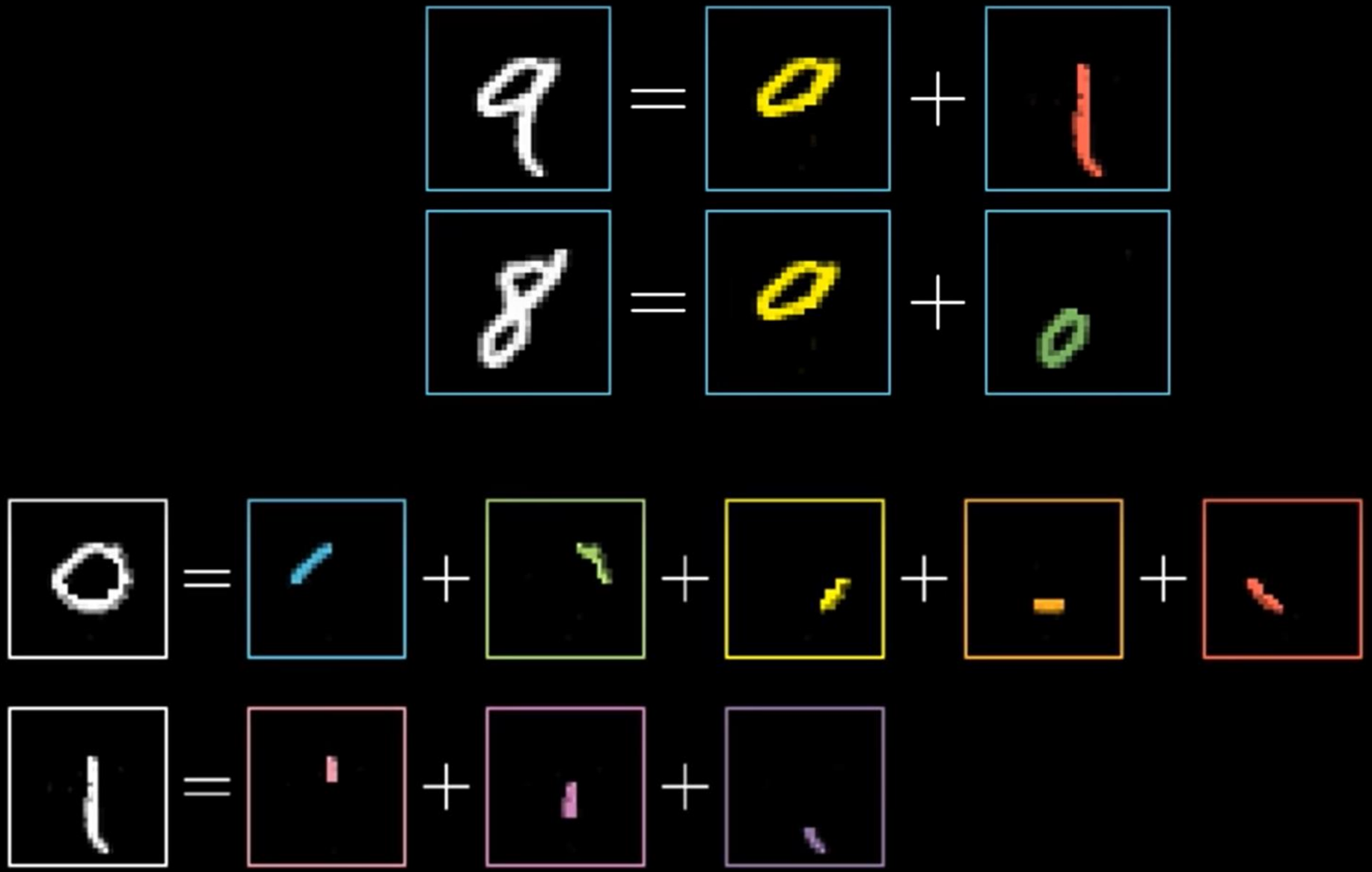


- The fundamental building blocks of neural networks
- Each has an activation function, modeled loosely after neurons in the brain, which "activate" when given enough stimulus (The human brain is estimated to have more than 10 billion neurons, to give you an idea)
- Can view a neuron graphically as a "node" with inputs, and weights

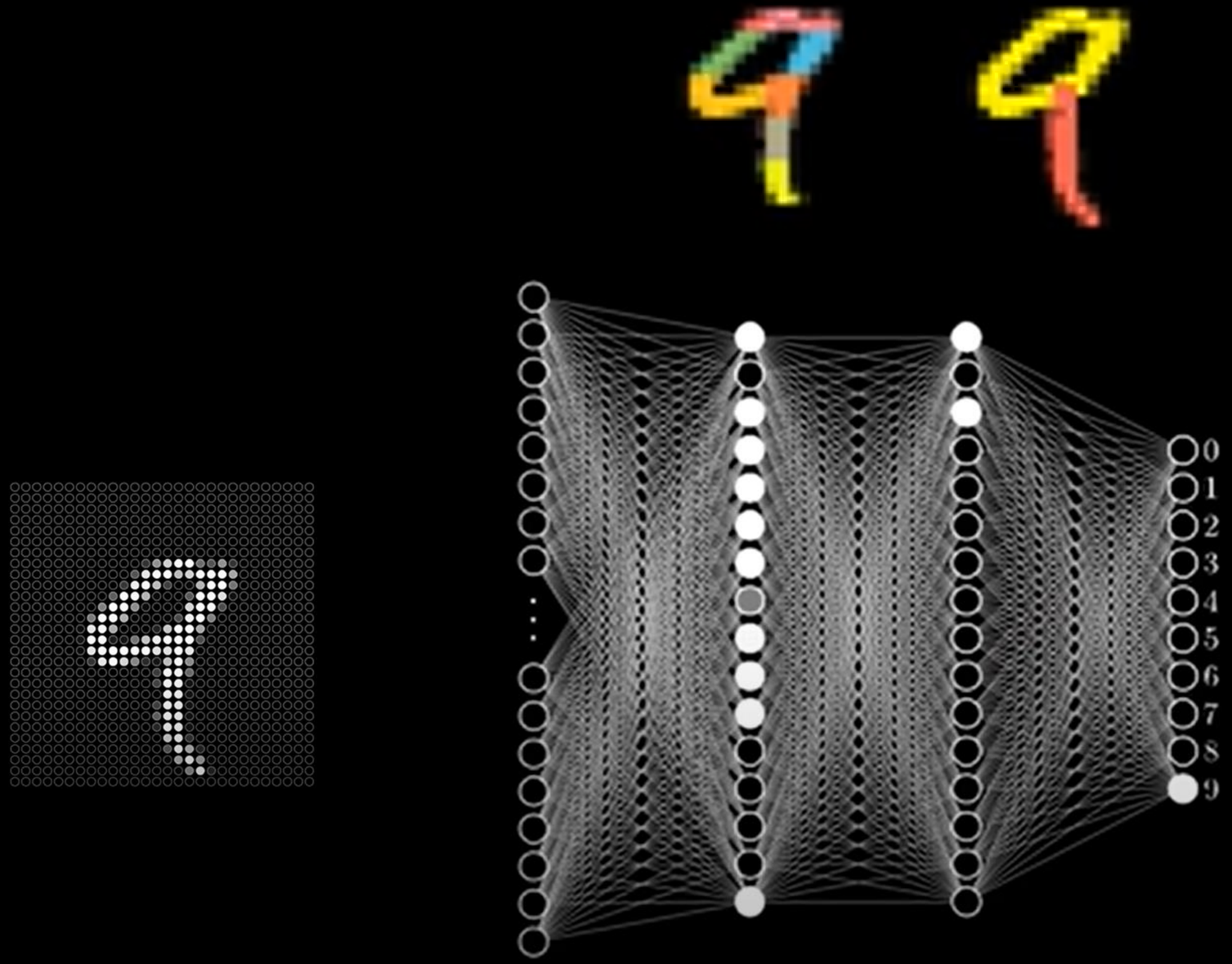




3Blue1Brown: But what is a neural network? | Chapter 1, Deep learning:
https://www.youtube.com/watch?v=aircAruvnKk&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi



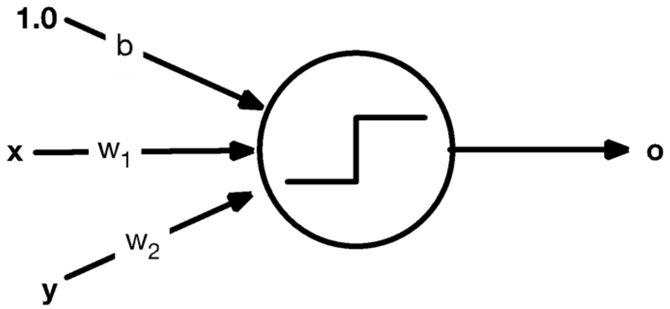
3Blue1Brown: But what is a neural network? | Chapter 1, Deep learning:
https://www.youtube.com/watch?v=aircAruvnKk&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi



3Blue1Brown: But what is a neural network? | Chapter 1, Deep learning:
https://www.youtube.com/watch?v=aircAruvnKk&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

Perceptrons

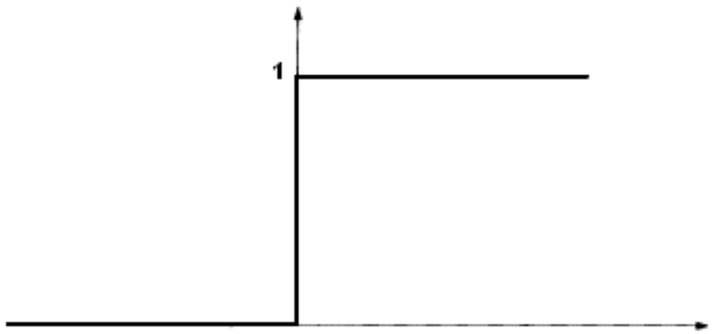
- The simplest form of a neuron
- Activation function is a unit step function: either ‘on’ or ‘off’
- It uses a linear decision boundary



$$sum = [b\ w_1\ w_2] \begin{bmatrix} 1.0 \\ x \\ y \end{bmatrix}$$

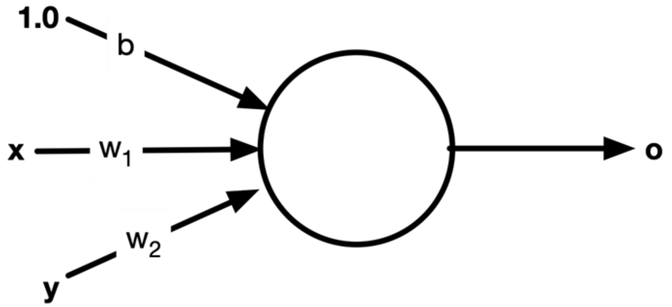
$$o = f(sum) = \begin{cases} 0, & sum \leq 0 \\ 1, & sum > 0 \end{cases}$$

Unit step (threshold)

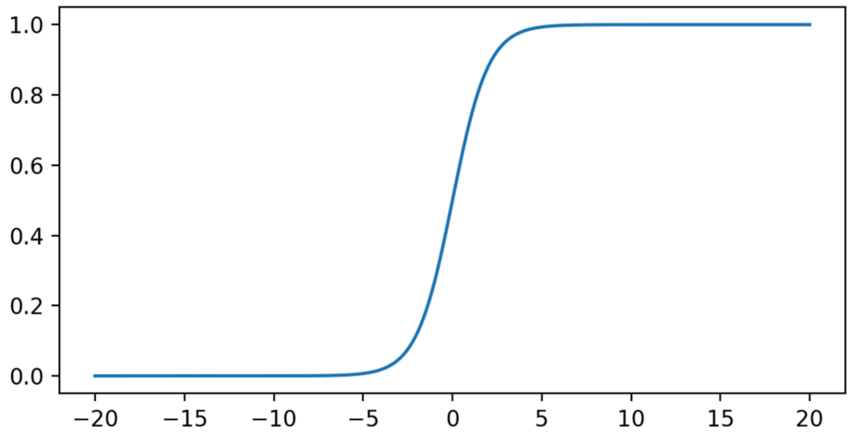


But the simple unit-step function doesn't work

- Training a neural network refers to finding the write modeling coefficients; b, w1, and w2.. which minimizes the error
- That means the activation function needs to be differentiable, and a simple unit-step function is not.



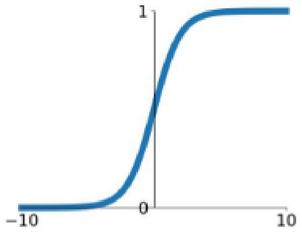
$$o = f(sum) = \frac{1}{1 + e^{-S}}$$



We use *pseudo*-unit-step functions

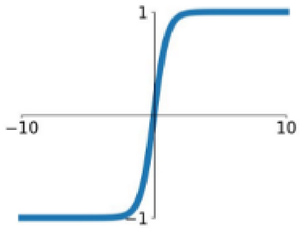
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



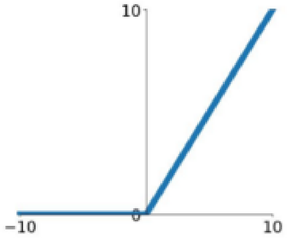
tanh

$\tanh(x)$



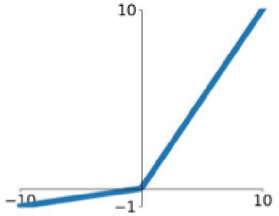
ReLU

$\max(0, x)$



Leaky ReLU

$\max(0.1x, x)$



- All of them are commonly-used

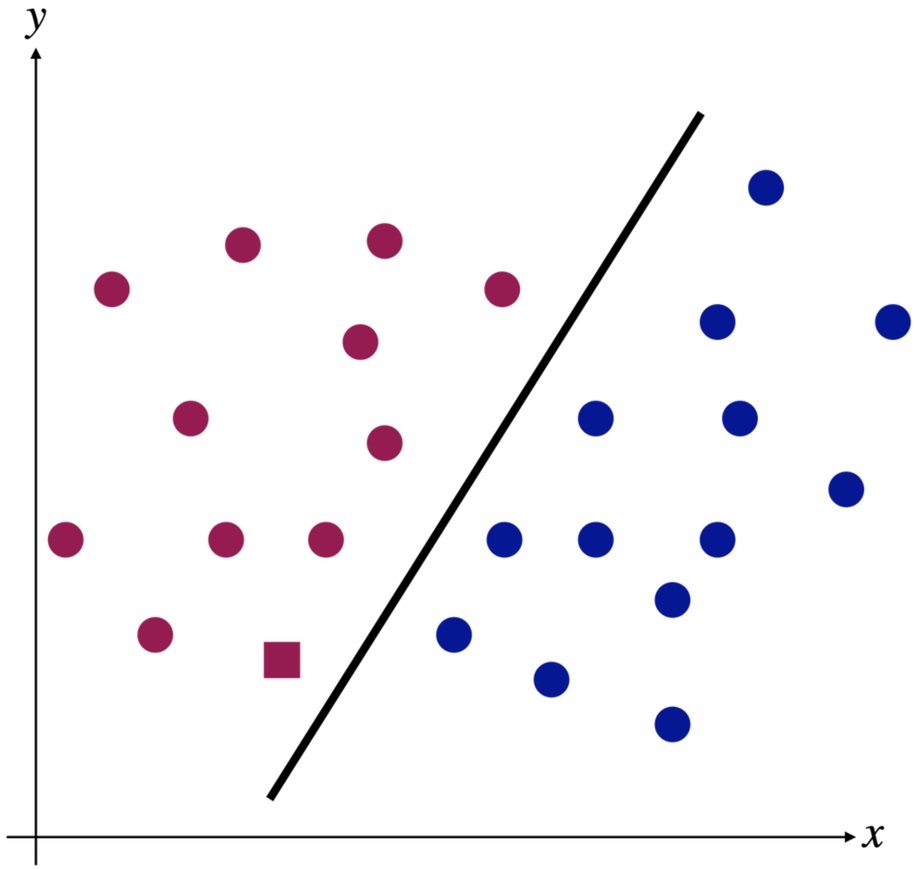
Question: What is a neural network with a single neuron whose activation function is Sigmoid?

Creating decisions with neurons

- Similar to other classification algorithms, neural networks also aims to define a decision boundary

- A perceptron will induce a linear (i.e. straight line) decision boundary

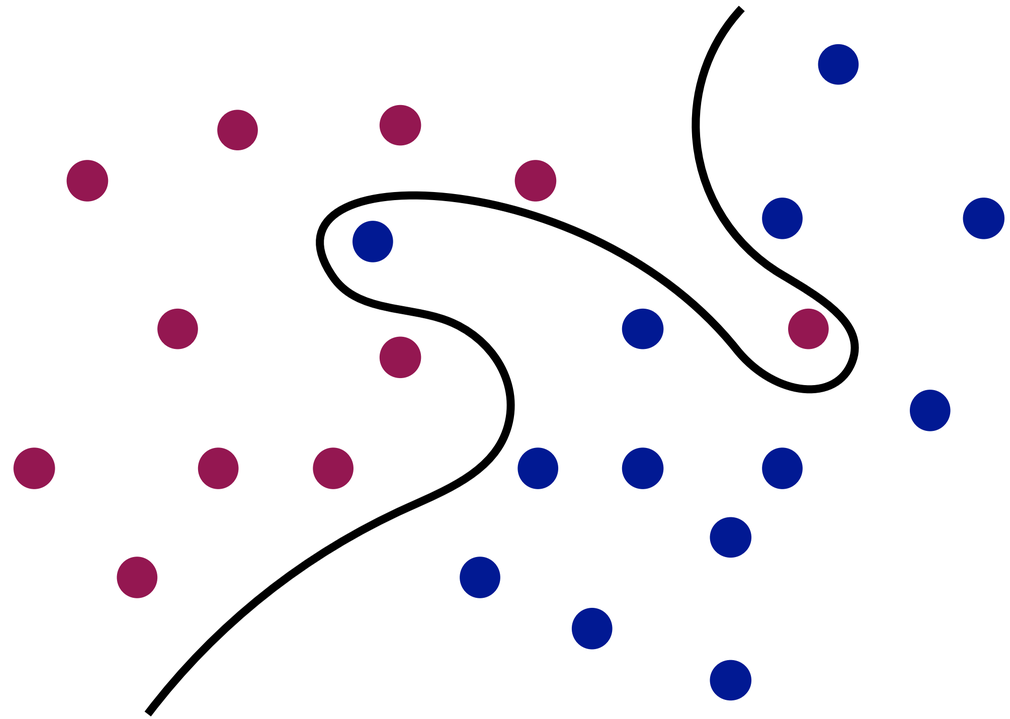
$$f(x,y) = \begin{cases} 0, & b + w_1 x + w_2 y \leq 0 \\ 1, & b + w_1 x + w_2 y > 0 \end{cases}$$



What if the decision boundary is non-linear?

- Add more layers of neurons so the model becomes not a straight line (i.e. equation has a higher degree)

- or a smoother activation (i.e. non-linear) function the resulting model becomes not a straight line



Multi-layer Neural Network

- The output from a neuron goes into other neurons in the next layer
- also called deep neural network (aka deep learning)

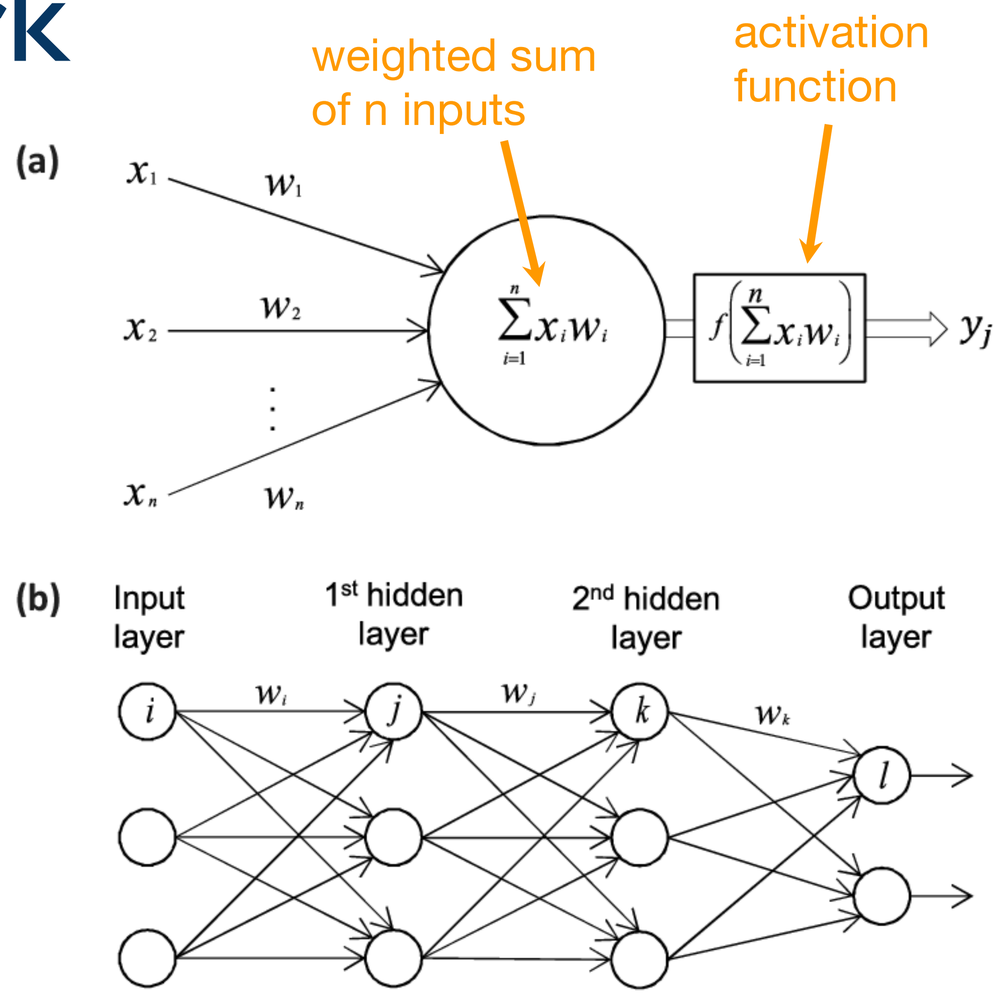


Figure from: Vieira, Sandra & Pinaya, Walter & Mechelli, Andrea. (2017). Using deep learning to investigate the neuroimaging correlates of psychiatric and neurological disorders: Methods and applications. Neuroscience & Biobehavioral Reviews. 74. 10.1016/j.neubiorev.2017.01.002.

Deep NN in Mathematical Form

- Outputs from the neurons at the j layer (j1, j2, and j3):

$$sum_{j1} = w_{11}x_1 + w_{21}x_2 + \cdots + w_{n1}x_n = \sum_1^n w_{i1}x_i = \boldsymbol{w}_{j1}^T X$$

$$sum_{j2} = w_{12}x_1 + w_{22}x_2 + \cdots + w_{n2}x_n = \sum_1^n w_{i2}x_i = \boldsymbol{w}_{j2}^T X$$

$$sum_{jn} = w_{1n}x_1 + w_{2n}x_2 + \cdots + w_{nn}x_n = \sum_1^n w_{in}x_i = \boldsymbol{w}_{jn}^T X$$

- It can be summarized as

$$sum_j = \begin{bmatrix} \boldsymbol{w}_{j1}^T \\ \boldsymbol{w}_{j2}^T \\ \vdots \\ \boldsymbol{w}_{jn}^T \end{bmatrix} X = W_j X$$

linear transformation

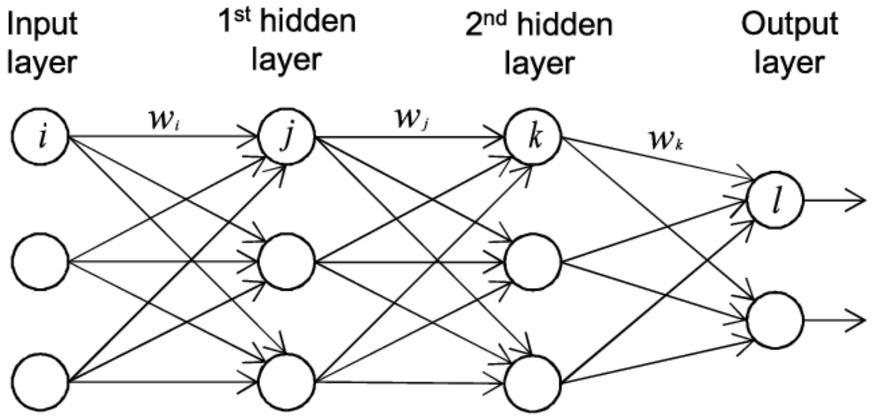


Figure from: Vieira, Sandra & Pinaya, Walter & Mechelli, Andrea. (2017). Using deep learning to investigate the neuroimaging correlates of psychiatric and neurological disorders: Methods and applications. Neuroscience & Biobehavioral Reviews. 74. 10.1016/j.neubiorev.2017.01.002.

Deep NN in Mathematical Form

- Then apply the activation function to each output (sum_{j1}, sum_{j2}, … , sum_{jn})

$$Y_j = \begin{bmatrix} f(sum_{j1}) \\ f(sum_{j2}) \\ \vdots \\ f(sum_{jn}) \end{bmatrix} = f(\boldsymbol{sum}_j) = f(W_j X)$$

non-linear transformation

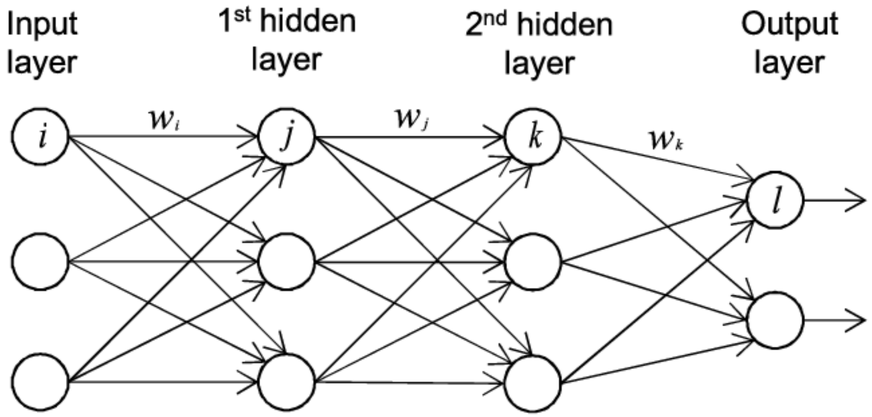


Figure from: Vieira, Sandra & Pinaya, Walter & Mechelli, Andrea. (2017). Using deep learning to investigate the neuroimaging correlates of psychiatric and neurological disorders: Methods and applications. Neuroscience & Biobehavioral Reviews. 74. 10.1016/j.neubiorev.2017.01.002.

Deep NN in Mathematical Form

- Repeat this process for each layer until the output layer

$$\begin{bmatrix} Output_1 \\ Output_2 \end{bmatrix} = f(W_l f(W_k f(W_j X)))$$

alternate linear and non-linear transformation => non-linear transformation!

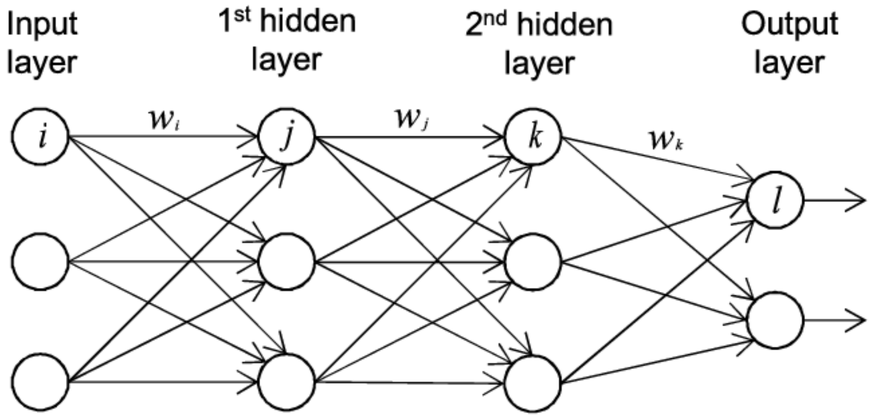
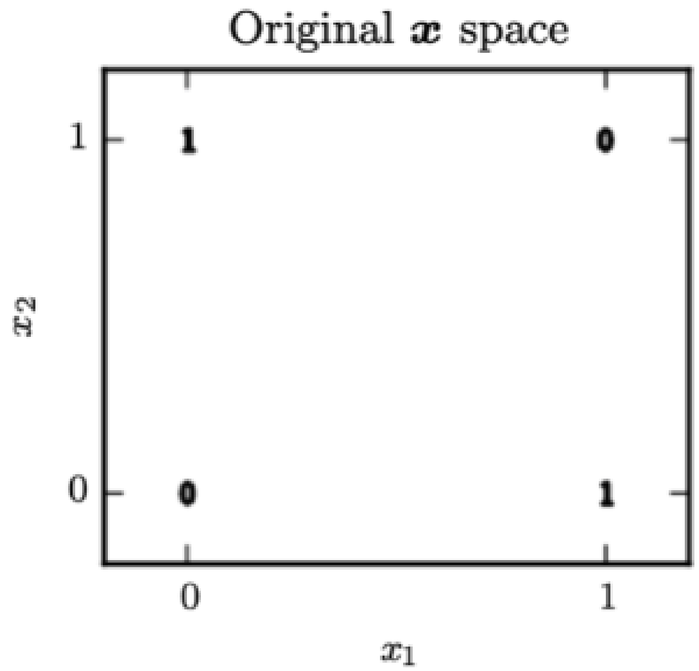


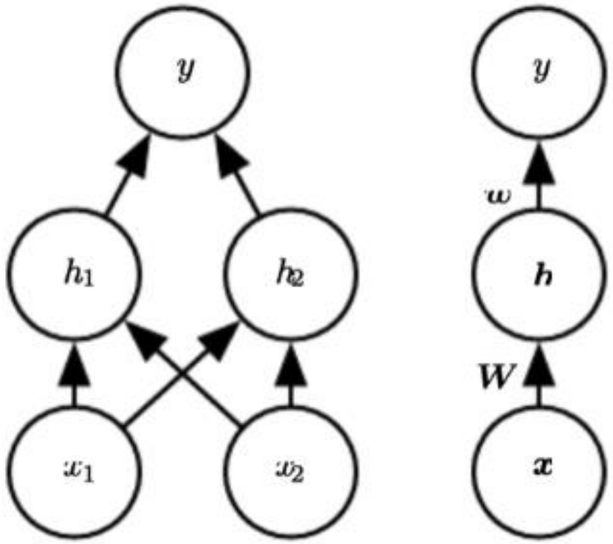
Figure from: Vieira, Sandra & Pinaya, Walter & Mechelli, Andrea. (2017). Using deep learning to investigate the neuroimaging correlates of psychiatric and neurological disorders: Methods and applications. Neuroscience & Biobehavioral Reviews. 74. 10.1016/j.neubiorev.2017.01.002.

Non-linear Boundary Example



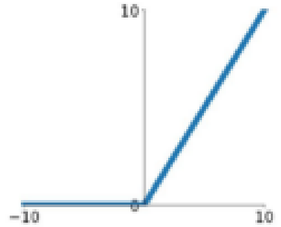
Non-linear Boundary Example

- two layer NN with the activation function ReLU



ReLU

$\max(0, x)$



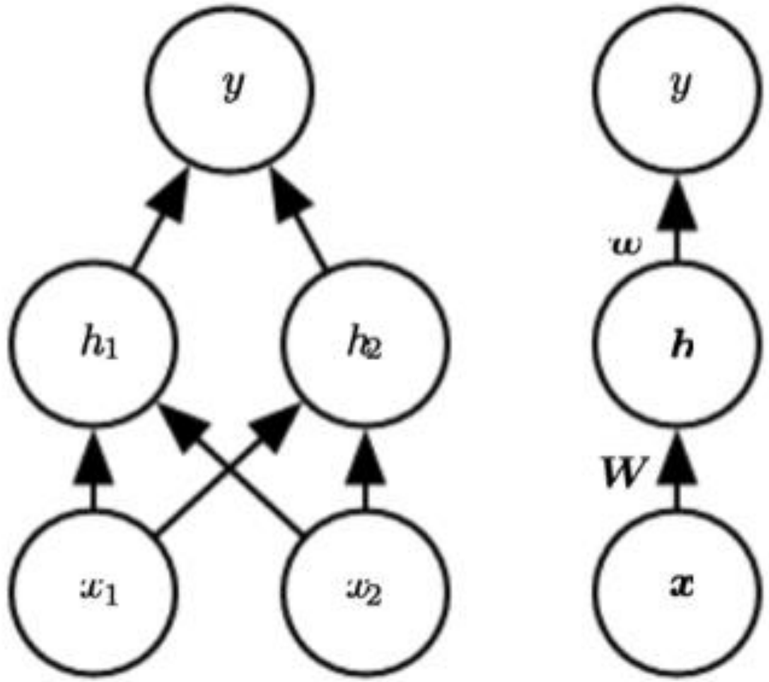
Non-linear Boundary Example

- two layer NN with the activation function ReLU

$$y = \boldsymbol{w}_y^T H$$

$$H = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} f(sum_{h1}) \\ f(sum_{h2}) \end{bmatrix} = \begin{bmatrix} f(\boldsymbol{w}_{h1} X + c_{h1}) \\ f(\boldsymbol{w}_{h2} X + c_{h2}) \end{bmatrix}$$

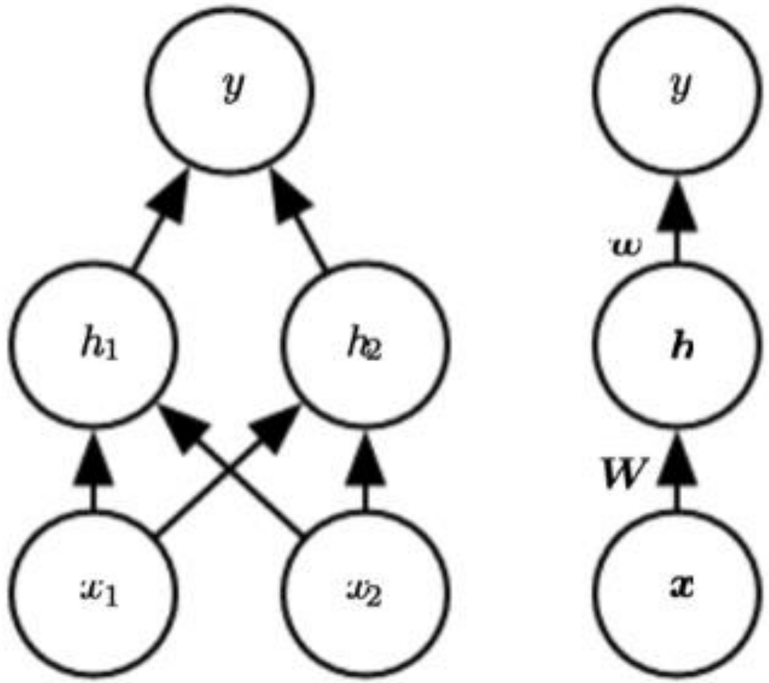
$$= f(WX + C) = \max\{0, WX + C\}$$



Non-linear Boundary Example

- Solution/Trained coefficients are:

$$W = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, c = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \boldsymbol{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

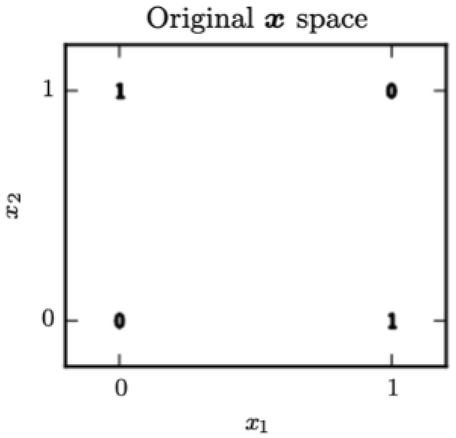


Walkthrough

$$H = f(WX + C) = \max\{0, WX + C\}$$

$$y = \boldsymbol{w}_y^T H$$

$$W = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, C = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \boldsymbol{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$$



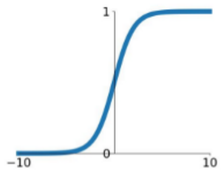
- We can verify that this two-layer NN implements the XOR function:
- $x_1 = 0$ and $x_2 = 0$: $\mathbf{h} = ReLU\left(\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}\begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix}\right) = ReLU\left(\begin{bmatrix} 0 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, y = \begin{bmatrix} 1 & -2 \end{bmatrix}\begin{bmatrix} 0 \\ 0 \end{bmatrix} = 0$
- $x_1 = 0$ and $x_2 = 1$: $\mathbf{h} = ReLU\left(\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}\begin{bmatrix} 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix}\right) = ReLU\left(\begin{bmatrix} 1 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, y = \begin{bmatrix} 1 & -2 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1$
- $x_1 = 1$ and $x_2 = 0$: $\mathbf{h} = ReLU\left(\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix}\right) = ReLU\left(\begin{bmatrix} 1 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, y = \begin{bmatrix} 1 & -2 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1$
- $x_1 = 1$ and $x_2 = 1$: $\mathbf{h} = ReLU\left(\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}\begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix}\right) = ReLU\left(\begin{bmatrix} 2 \\ 1 \end{bmatrix}\right) = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, y = \begin{bmatrix} 1 & -2 \end{bmatrix}\begin{bmatrix} 2 \\ 1 \end{bmatrix} = 0$

Architecture and Parameters of NN

- **Depth**: # of layers
- **Width**: # of neurons per layer
- **Activations**: sigmoid, ReLU, tanh, etc.

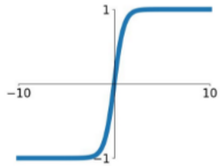
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



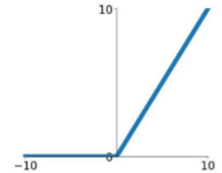
tanh

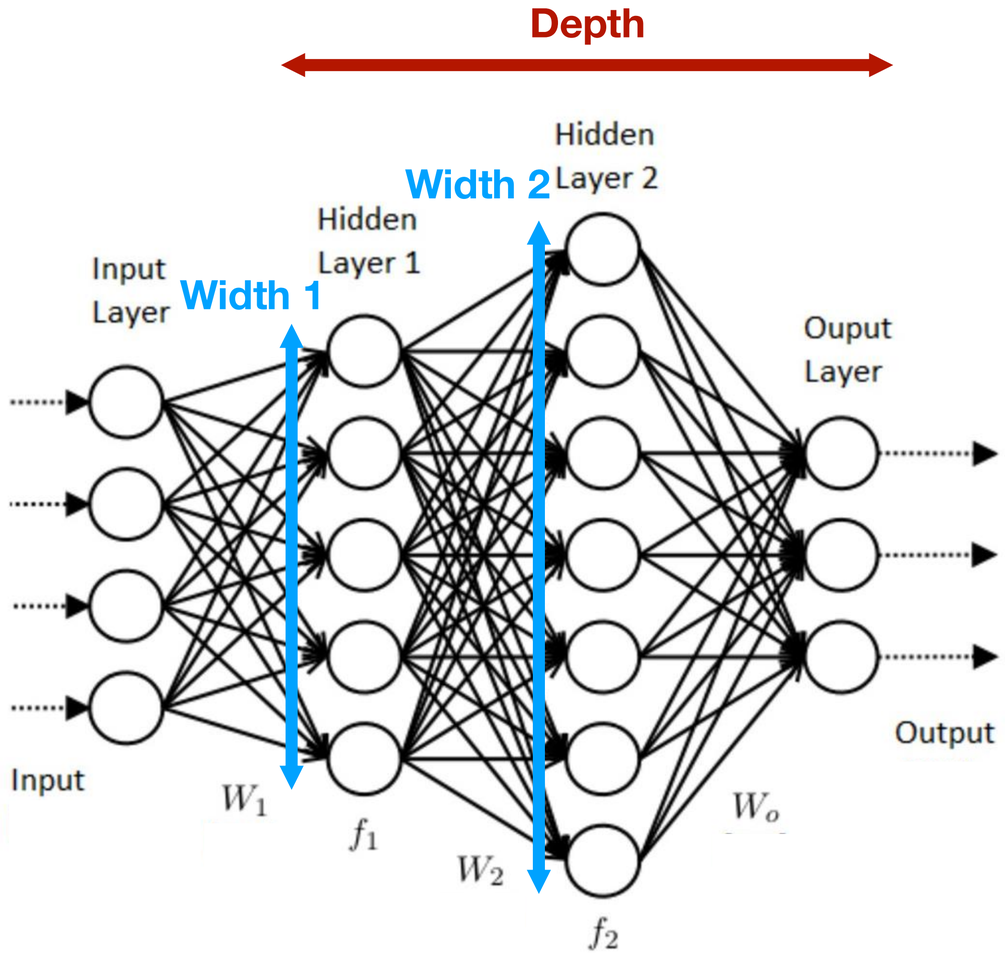
$\tanh(x)$



ReLU

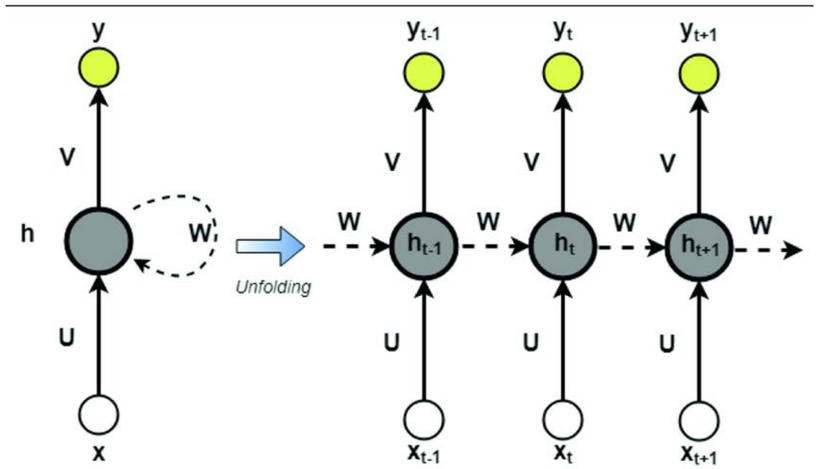
$\max(0, x)$

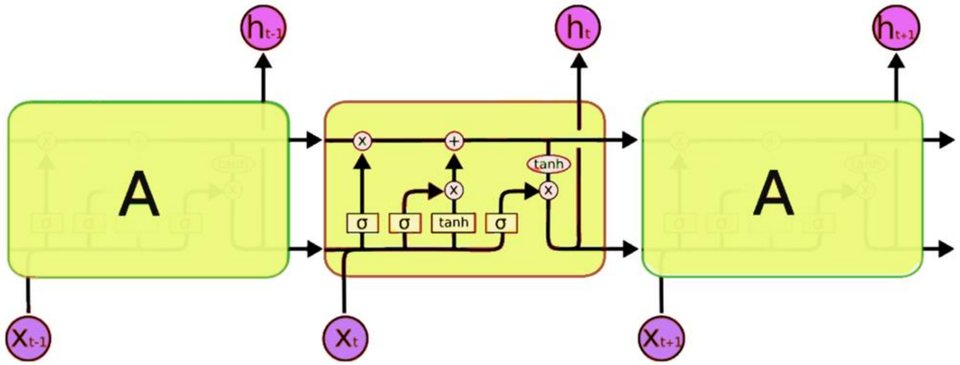




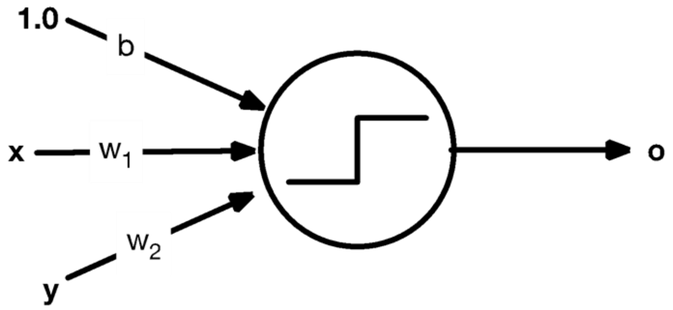
There are a variety of NNs out there!

- A plethora of neural network architectures have been proposed, for different applications
  - Multi-layer Perceptron (**MLP**): Cascading perceptrons
  - Recurrent Neural Networks (**RNN**): Sequential data modeling
  - Convolutional Neural Networks (**CNN**): Image recognition
  - Long Short Term Memory (**LSTM**): Memory cells with “forgetting” factors
  - Transformer (most recent), Gated Recurrent Units (**GRU**), Hopfield Networks, Boltzmann Machines, Generative Adversarial Networks (**GAN**), …

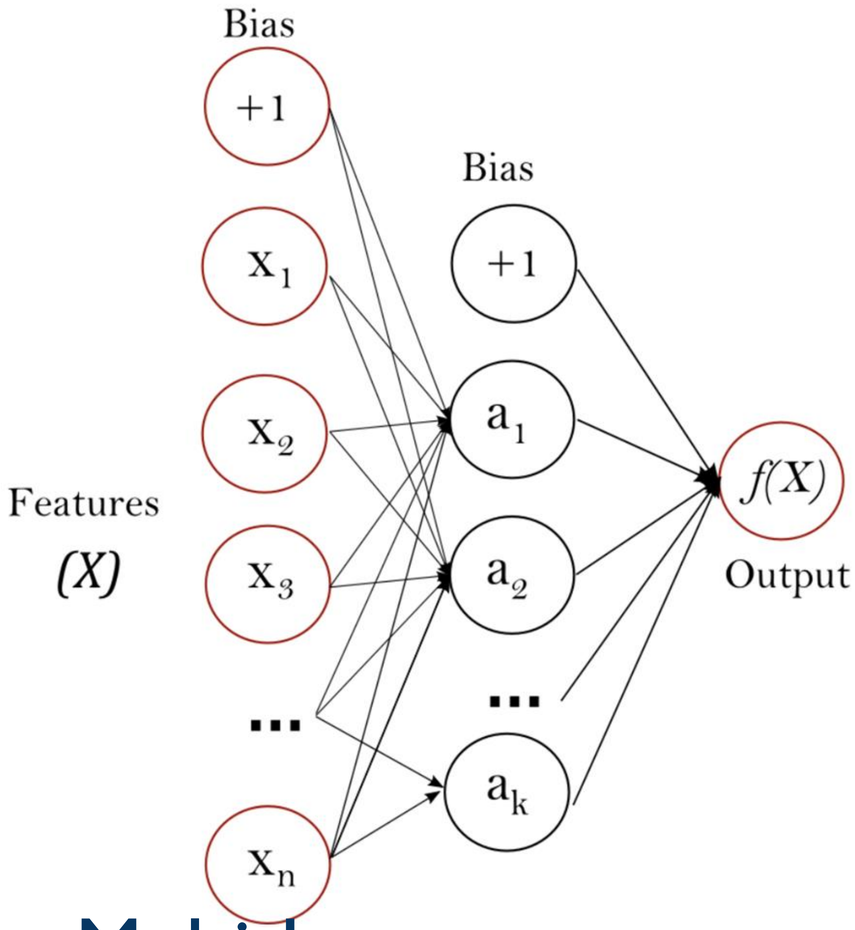




What we learned so far is the closest to MLP



A single perceptron



Multi-Layer Perceptron

Implementing neural networks

- sklearn has a built in MLP module:

```
from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(hidden_layer_sizes=(13,13,13),max_iter=500)
```

- For more complex neural networks, we typically leverage other machine learning libraries/platforms:
  - pytorch (https://pytorch.org/)
  - tensorflow (https://www.tensorflow.org/)
  - Both have Python interfaces

TensorFlow

PyTorch

MLP in Python

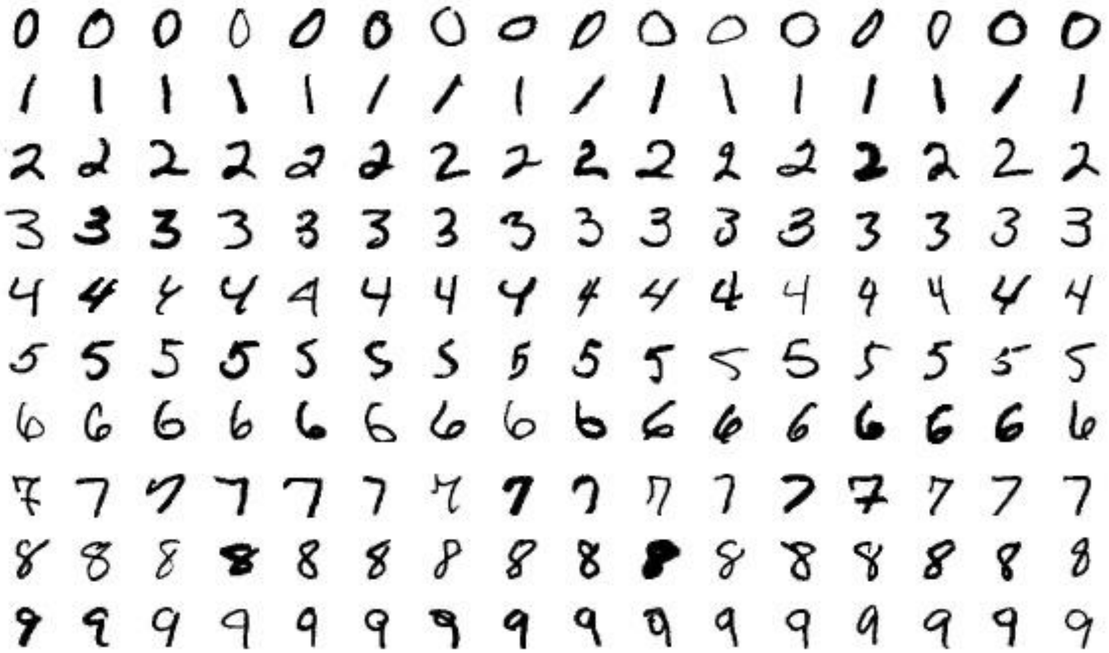
```
from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(hidden_layer_sizes=(13,13,13), max_iter=500)

mlp.fit(train_X, train_y)

print(mlp.coefs_[i])  # weight matrix corresponding to layer i (i=0,...,3)

print(mlp.intercept_[i])  # bias vector for neurons in layer i+1 (i=0,...,2)

results = mlp.predict(test_X)
```

three hidden layers, each with 13 neurons

https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

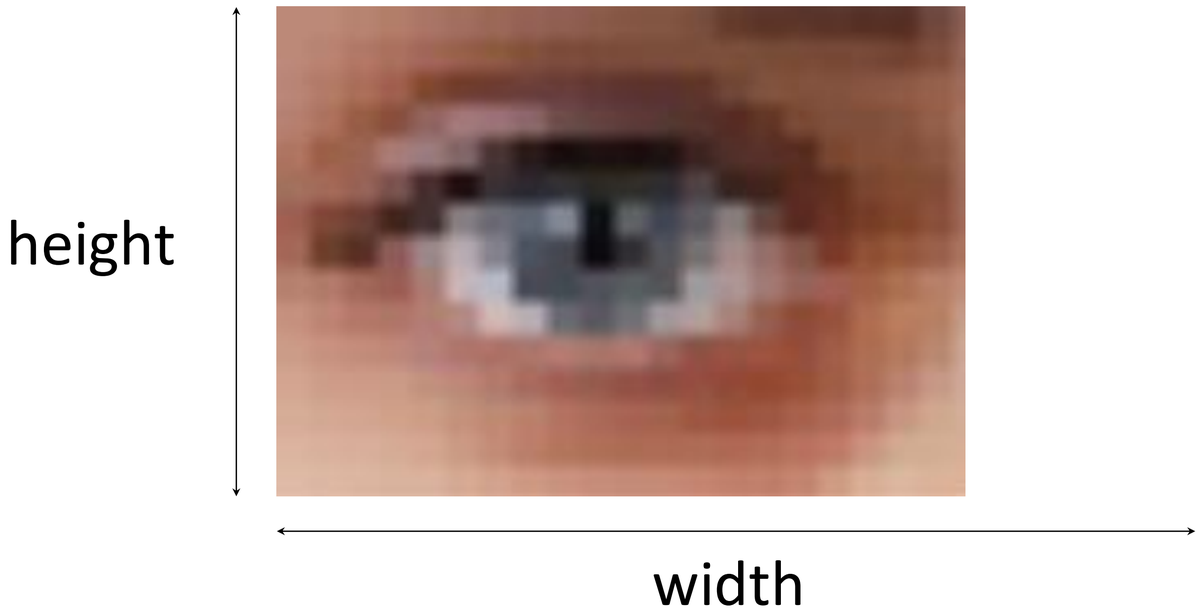
Let’s classify numeric characters!

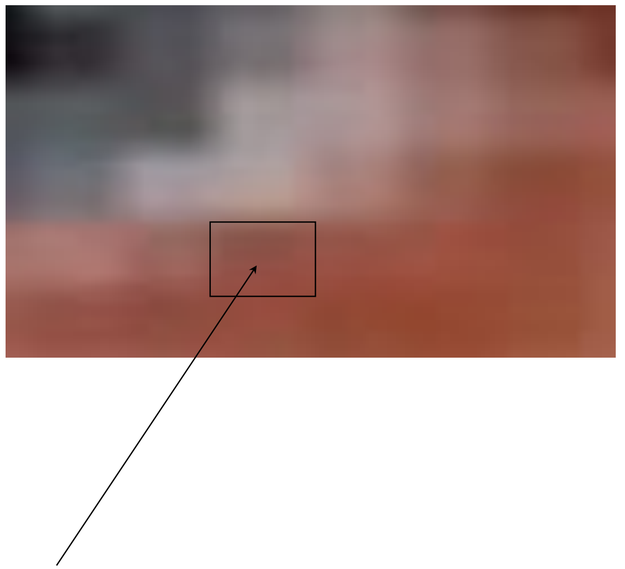
- MNIST dataset



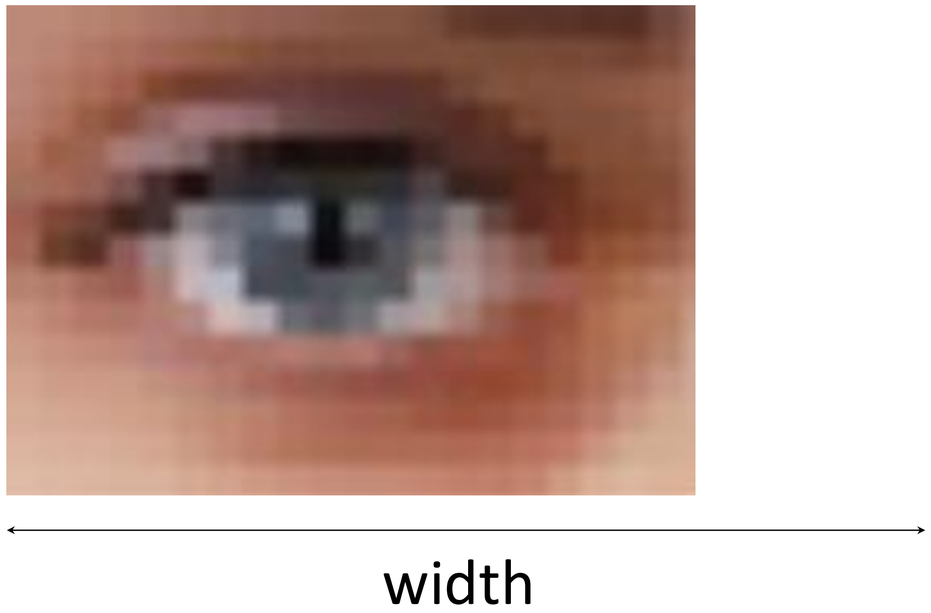
- 70,000 images of handwritten digits
- Each image is a grayscale 28 x 28 resolution image
- Each image belongs to one of ten classes: 0, 1, 2, 3, 4, 5, 6, 7, 8, or 9
- The classification task, when using this dataset, is to predict the handwritten digit in the image

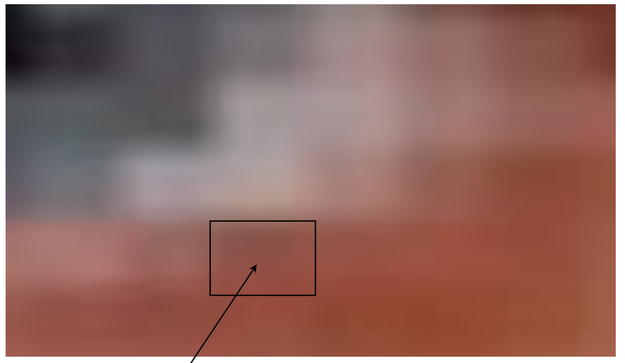
How are images represented on a computer?





How are images represented on a computer?





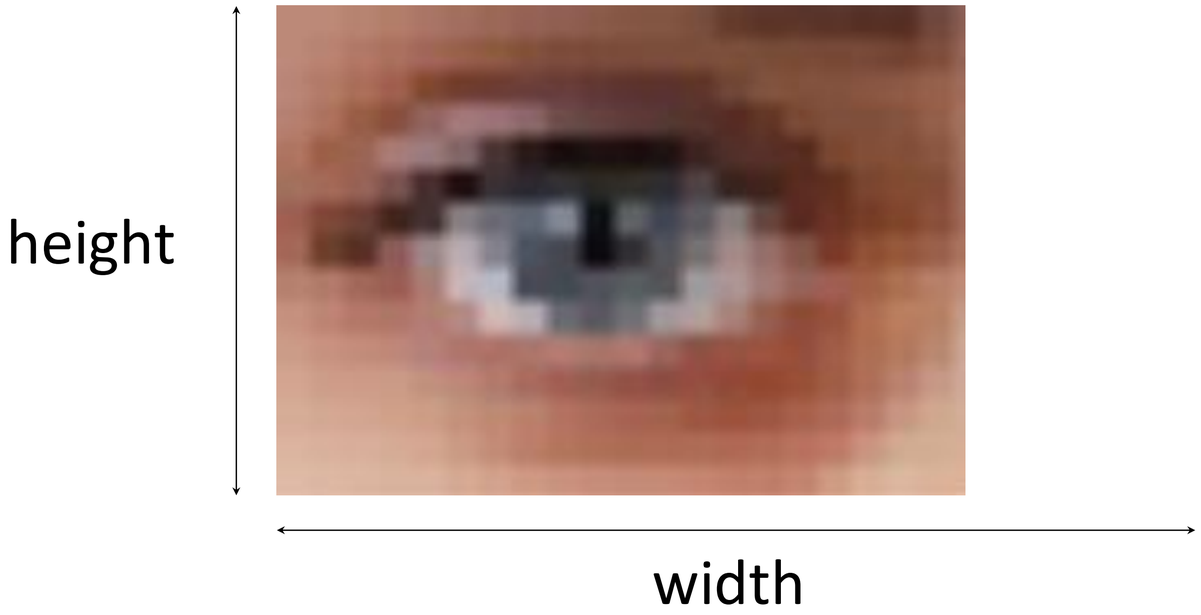
Pixel

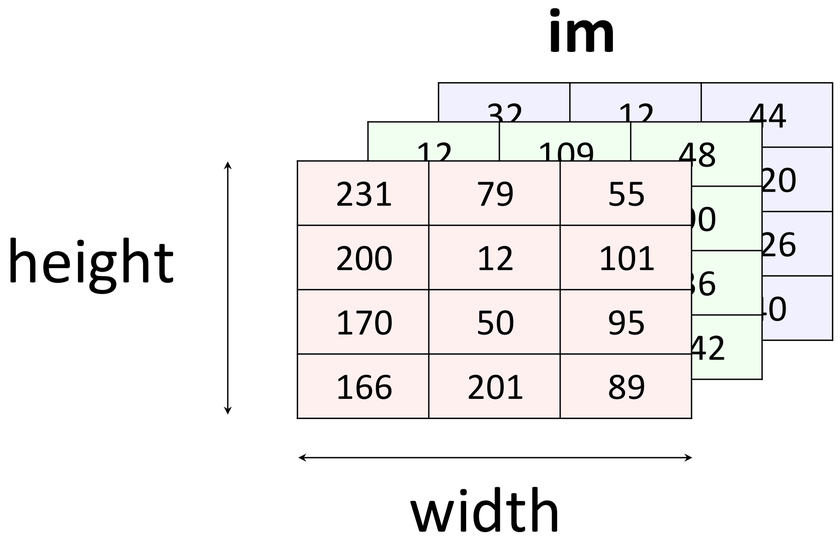
R: 145

G: 81

B: 69

How are images represented on a computer?



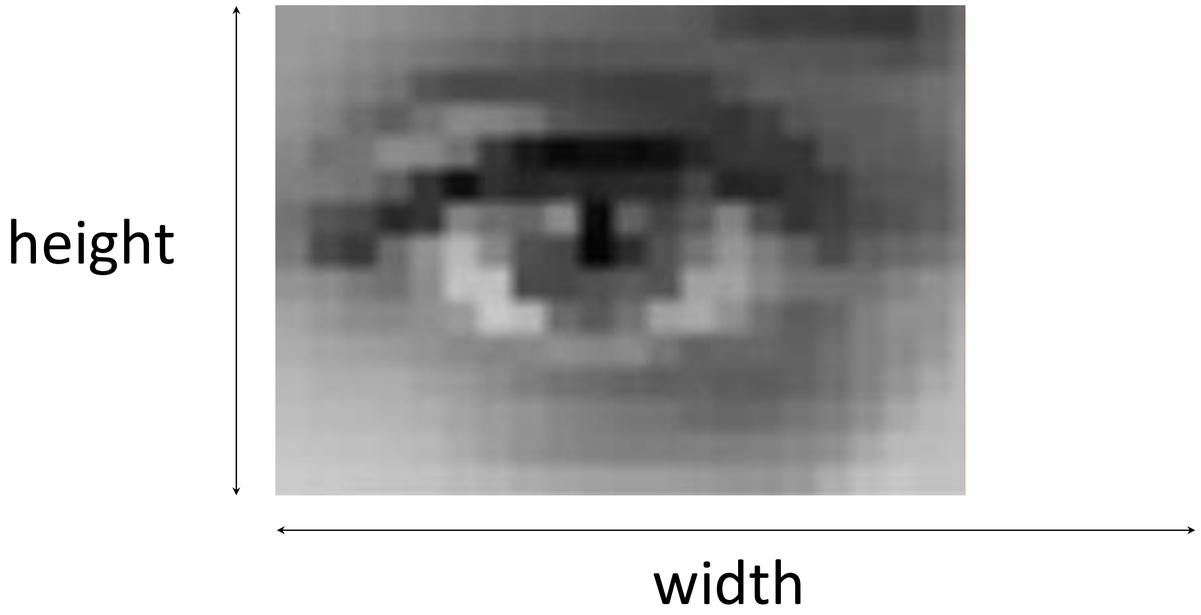


im.shape
(4, 3, 3)

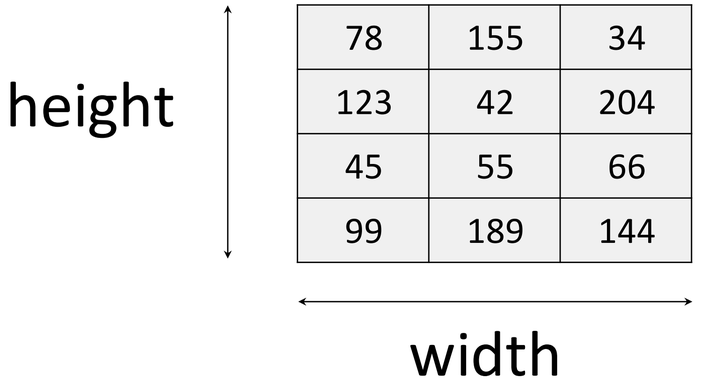
3D array with: row# column# RGB values

How are images represented on a computer?

Grayscale images …



im_gray



im_gray.shape

(4, 3)

row column

What are my input features?

image 1

78	155	34
123	42	204
45	55	66
99	189	144

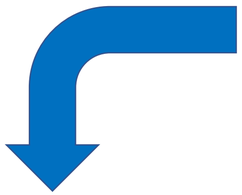
image 2

0	155	0
0	155	0
0	155	0
10	155	0

… (more images)

	x_1	x_2	x_3	x_4			…				x_{11}	x_{12}
image 1	78	155	34	123	42	204	…	…	…	99	189	144
image 2	0	155	0	0	155	0	…	…	…	10	155	0
…												

Data pre-processing



im_gray

height

78	155	34
123	42	204
45	55	66
99	189	144

width

im_gray_flat

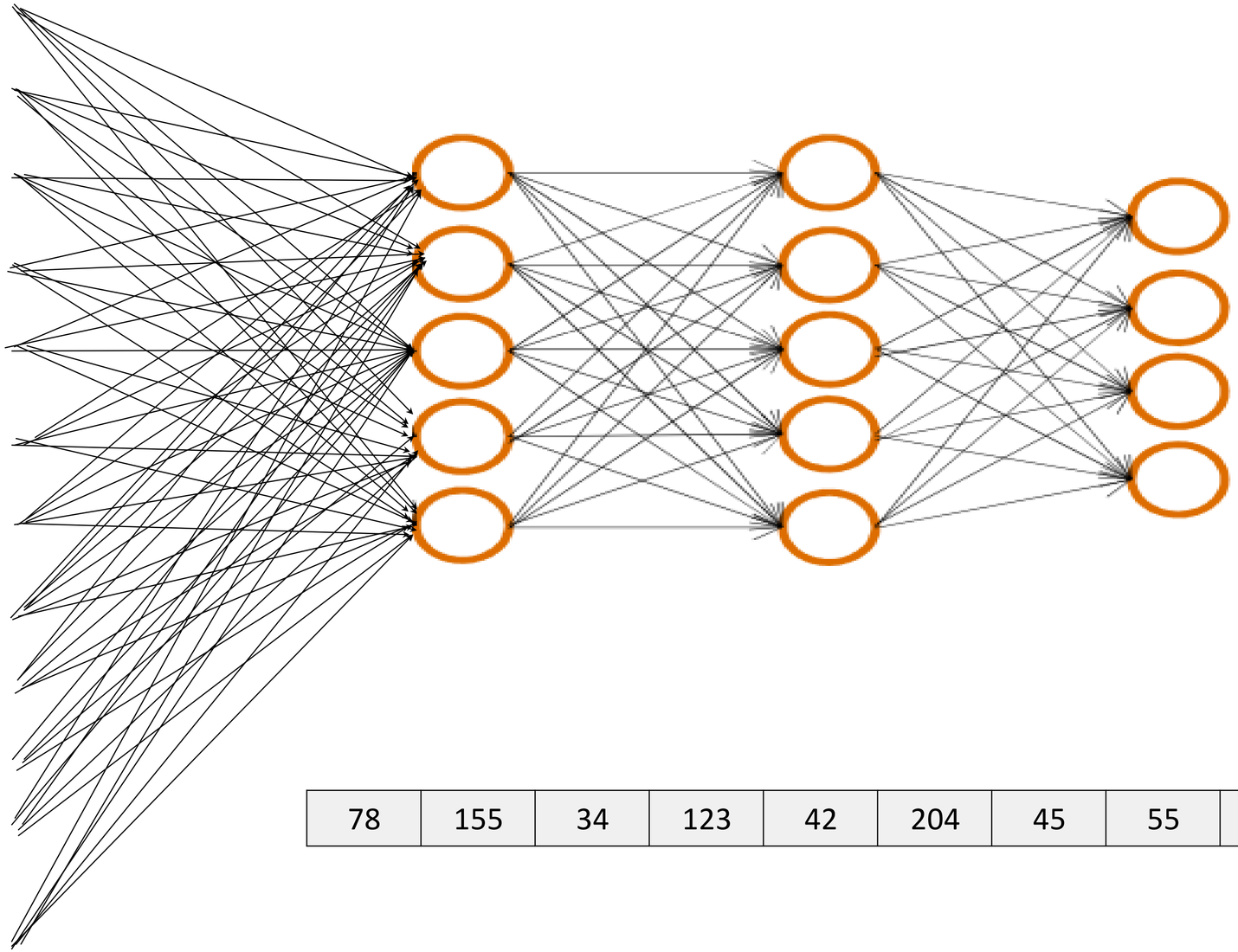
78	155	34	123	42	204	45	55	66	99	189	144

```
im_gray_flat = im_gray.reshape(1,12)

# This does the same: -1 auto-calculates the length
im_gray_flat = im_gray.reshape(1,-1)
```

im_gray_flat.shape
(1, 12)

Remember: this is a 2D numpy array

Now ready to build a NN



78	155	34	123	42	204	45	55	66	99	189	144